

SDITO-LAB

Name : Diptadeep Ghosh

Roll No : CSE/23/080

Exam RollNo: 11600223051

Assignment NO : 10

Problem Statement : Deploy a project from GitHub to EC2 by creating a new Security Group and configuring User Data.

AIM

To launch an EC2 instance, create and configure a new Security Group, deploy a project from GitHub using User Data script, and access the deployed application via public IP.

Prerequisites

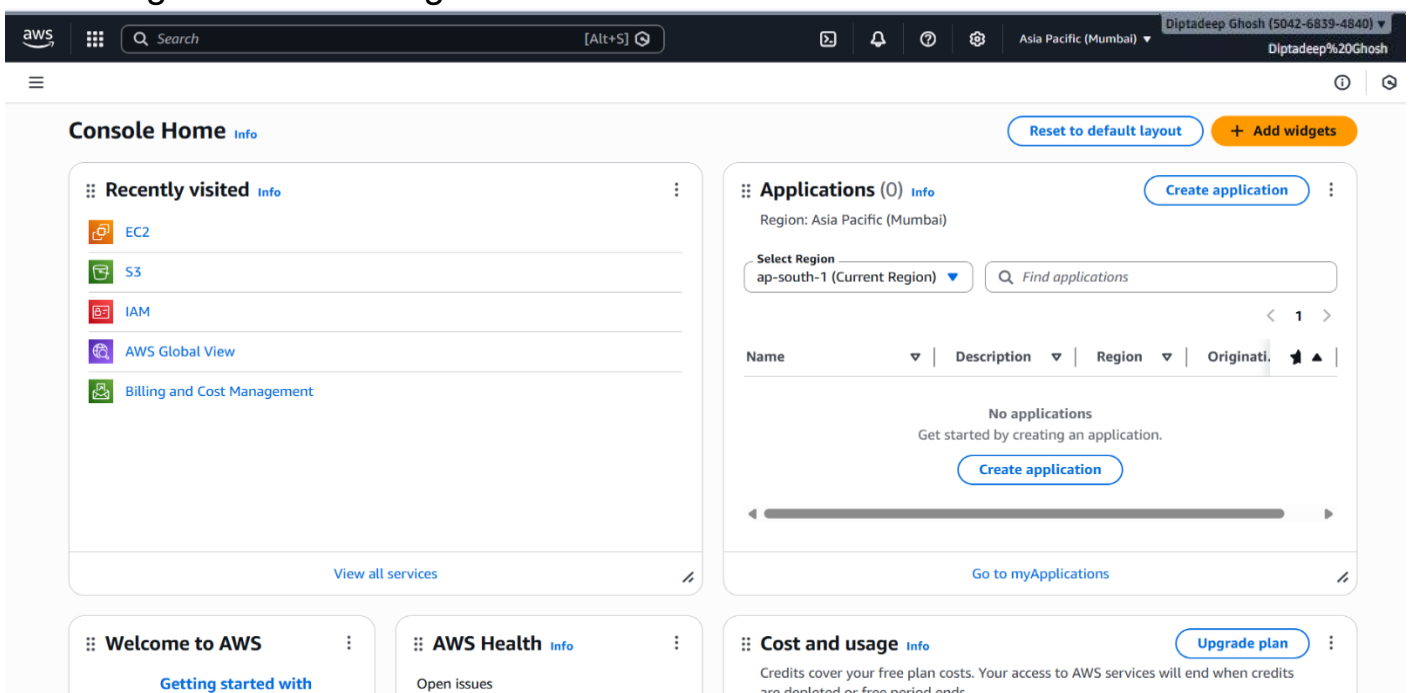
- AWS Account
- GitHub repository (public)
- Basic familiarity with Linux shell command, Git , and Node.js.
- Key Pair for EC2 login
- An AWS account with permission to create EC2 instances, security groups, and use EC2 Instance Connect.

Solution:

PHASE 1 – Create a Security Group

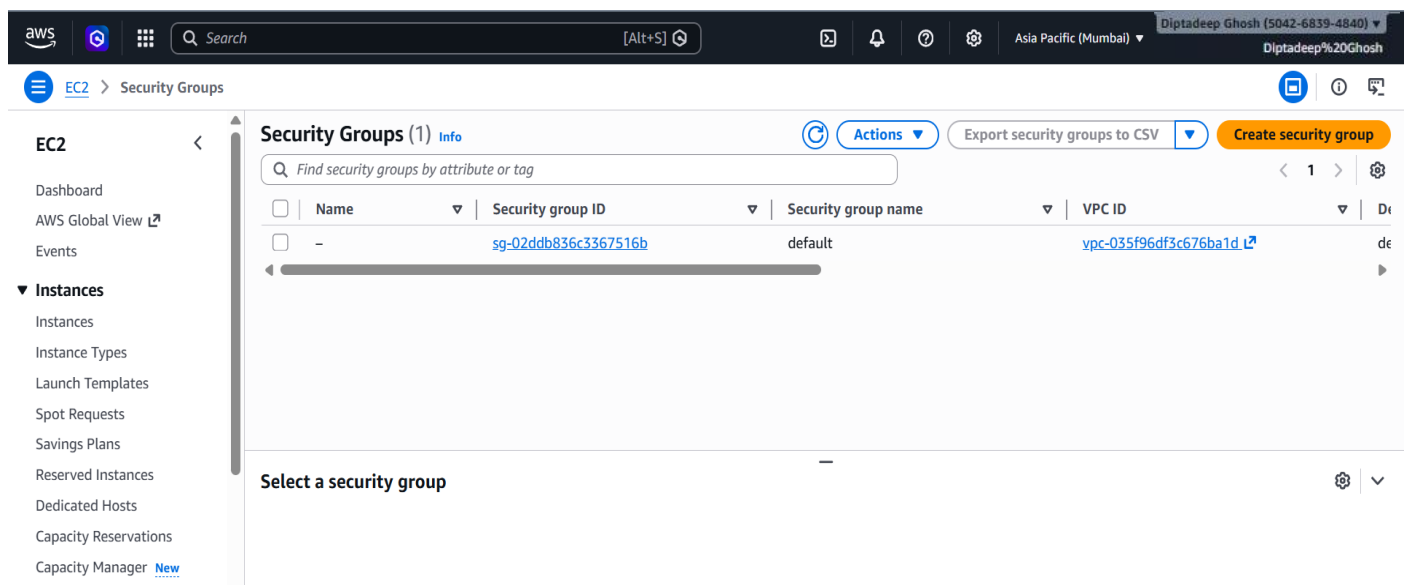
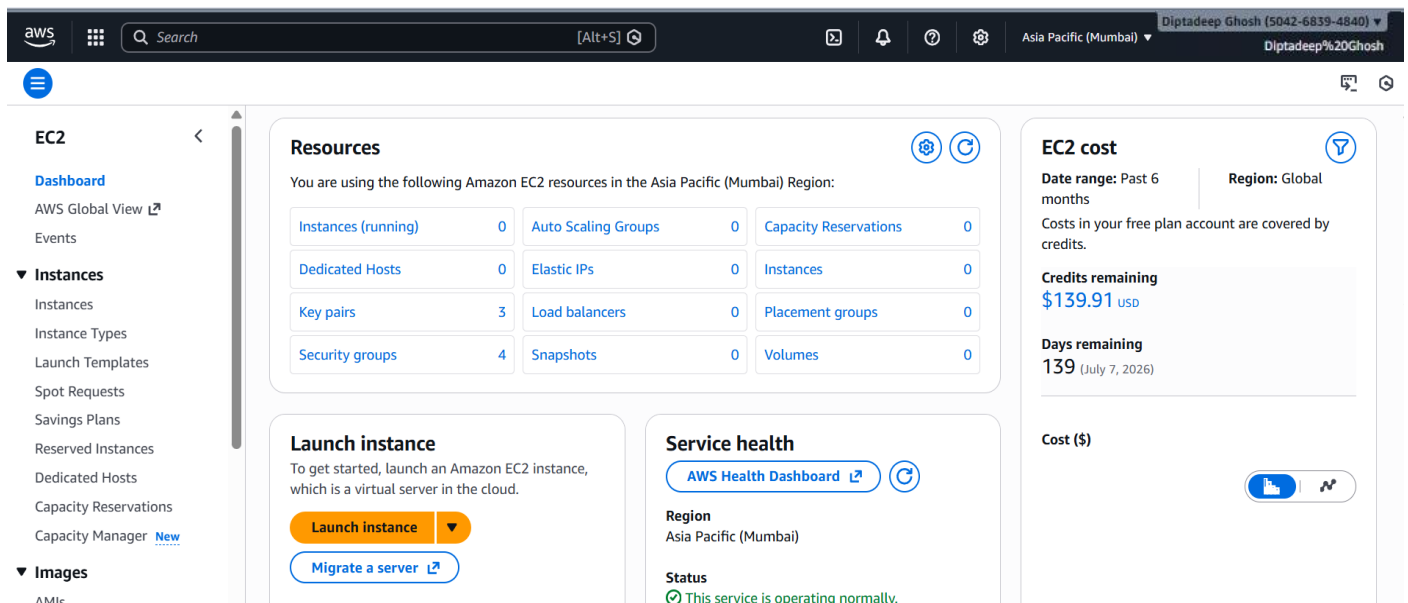
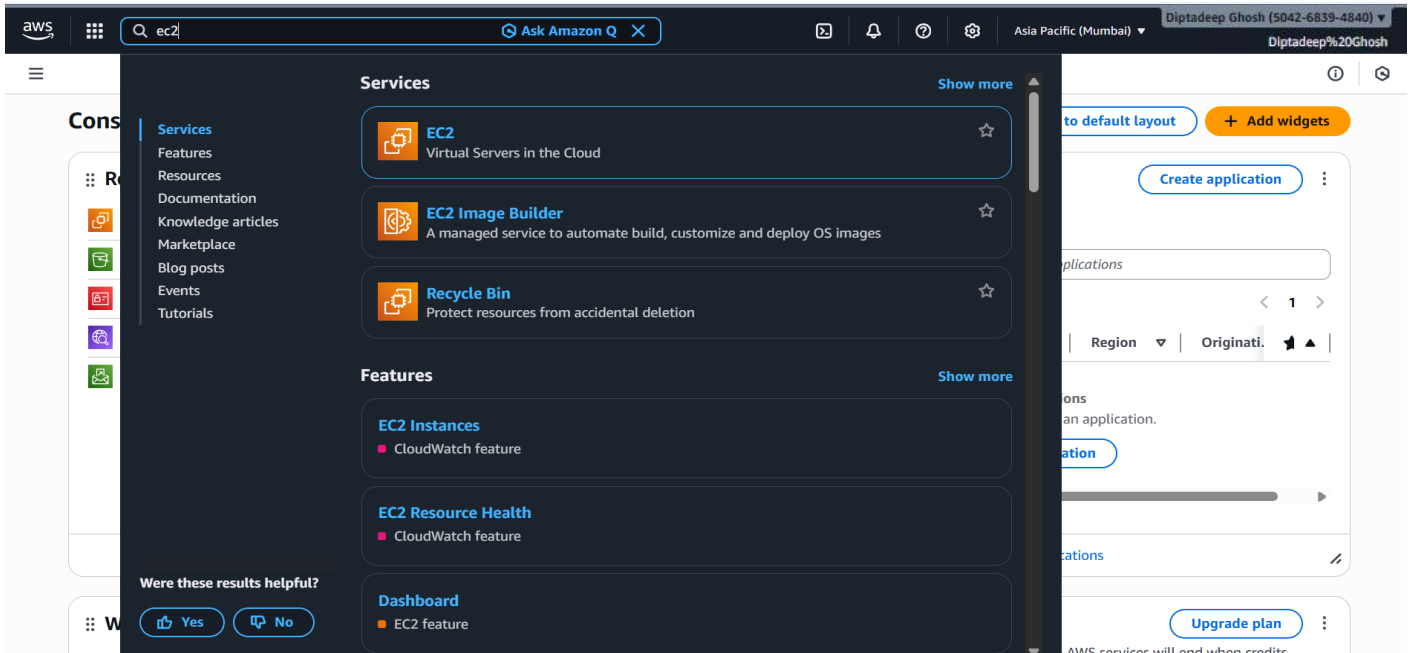
Step 1: Login to AWS Console

1. Login to AWS Management Console.



Step 2: Create a New Security Group

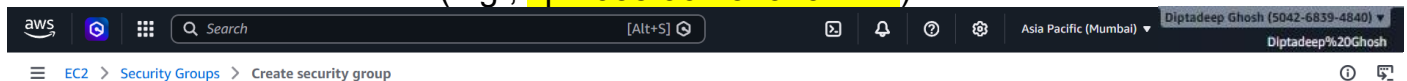
1. In the AWS Console, go to **EC2 > Network & Security > Security Groups**.



2. Click **Create security group**.

3. Fill basic details:

- **Security group name:** mysec110
- **Description:** mysec110
- **VPC:** select the VPC where you will launch the instance (e.g., vpc-035f96df3c676ba1d).



Create security group [Info](#)

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name [Info](#)

Name cannot be edited after creation.

Description [Info](#)

VPC [Info](#)

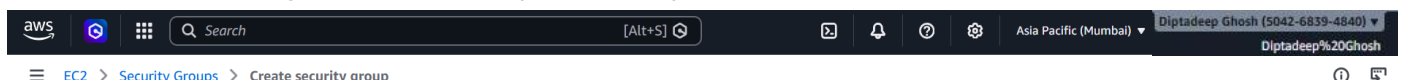
Inbound rules [Info](#)

This security group has no inbound rules.

[Add rule](#)

4. Click on **Add Inbound Rule** and **configure**.

- Add **SSH** (Type: **SSH**, Protocol: **TCP**, Port: **22**) — Source: **anywhere-ipv4** (0.0.0.0/0)
- Add **HTTP** (Type: **HTTP**, **TCP**, Port: **80**) — Source: **anywhere-ipv4** (0.0.0.0/0).
- Add **HTTPS** (Type: **HTTPS**, **TCP**, Port: **443**) — Source: **anywhere-ipv4** (0.0.0.0/0).
- Add **Custom TCP** (Type: **Custom TCP**, Protocol: **TCP**, Port range: **4000**) — Source: **anywhere-ipv4** (0.0.0.0/0).



Inbound rules [Info](#)

Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
SSH	TCP	22	Any...		Delete
HTTP	TCP	80	Any...	0.0.0.0/0	Delete
HTTPS	TCP	443	Any...	0.0.0.0/0	Delete
Custom TCP	TCP	4000	Any...	0.0.0.0/0	Delete

[Add rule](#)

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Outbound rules [Info](#)

Type Info	Protocol Info	Port range Info	Destination Info	Description - optional Info	
All traffic	All	All	Cust...		Delete

5. Configure **Outbound rules**: default is *All traffic* to 0.0.0.0/0. Leave as it is.
6. Now , Click on **Create security group**.

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags

Cancel

Create security group

- ❖ Now we can see a success message and the new **mysec110** security group is listed.

The screenshot shows the AWS Management Console interface. At the top, a green banner indicates that the security group 'sg-06ac5d8a6c5f7013b' (mysec110) was created successfully. Below this, the 'Details' tab is selected, showing the following information:

- Security group name:** mysec110
- Security group ID:** sg-06ac5d8a6c5f7013b
- Description:** mysec110
- VPC ID:** vpc-035f96df3c676ba1d
- Owner:** 780886633843
- Inbound rules count:** 4 Permission entries
- Outbound rules count:** 1 Permission entry

Below the details, there are tabs for 'Inbound rules', 'Outbound rules', 'Sharing', 'VPC associations', 'Related resources - new', and 'Tags'. The 'Inbound rules' tab is active, showing a table with 4 rules:

Name	Security group rule ID	IP version	Type	Protocol	Port range
-	sgr-04ab03de6b1f925f6	IPv4	HTTP	TCP	80
-	sgr-084d9502ff980118e	IPv4	HTTPS	TCP	443
-	sgr-005f2738e205e7cf5	IPv4	SSH	TCP	22
-	sgr-0f1f974ecfc2ddcb1	IPv4	Custom TCP	TCP	4000

The screenshot shows the AWS Management Console interface. At the top, a green banner indicates that the security group 'sg-06ac5d8a6c5f7013b' (mysec110) was created successfully. Below this, the 'Security Groups (1/2)' list is displayed. The table shows the following security groups:

Name	Security group ID	Security group name	VPC ID
-	sg-06ac5d8a6c5f7013b	mysec110	vpc-035f96df3c676ba1d
-	sg-02ddb836c3367516b	default	vpc-035f96df3c676ba1d

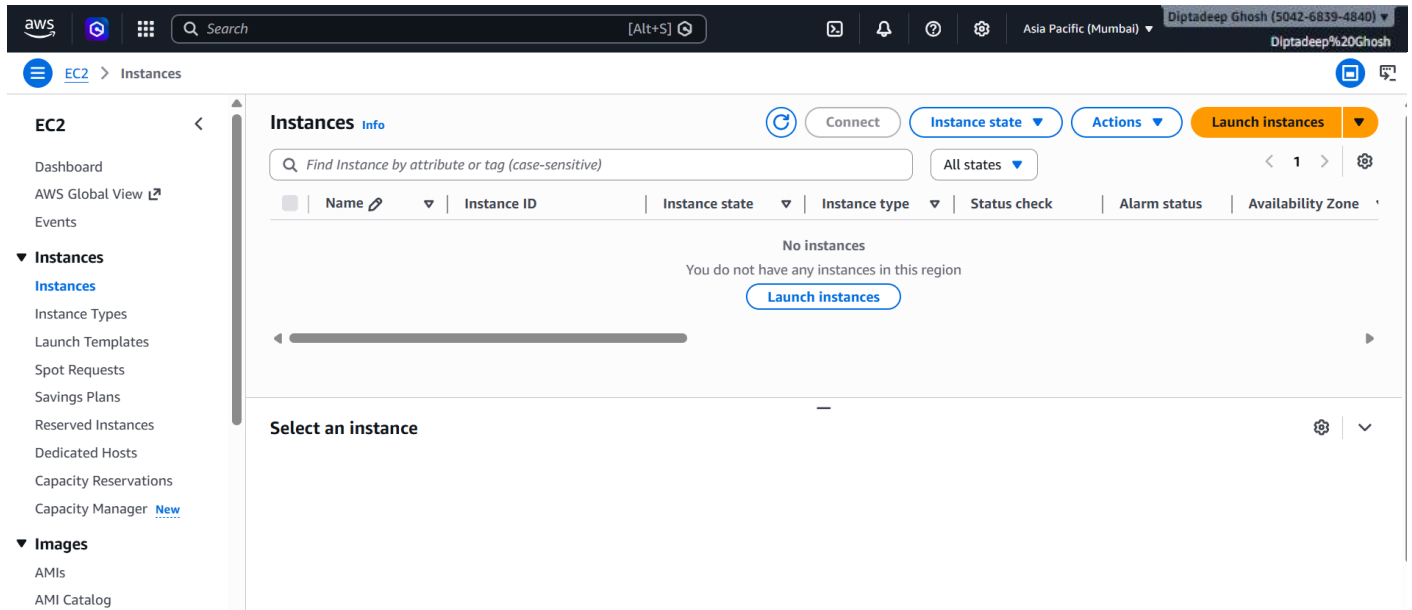
Below the list, the 'Details' tab is selected for the security group 'sg-06ac5d8a6c5f7013b - mysec110'. The details show the following information:

- Security group name:** mysec110
- Security group ID:** sg-06ac5d8a6c5f7013b
- Description:** mysec110
- VPC ID:** vpc-035f96df3c676ba1d
- Owner:** 780886633843
- Inbound rules count:** 4 Permission entries
- Outbound rules count:** 1 Permission entry

PHASE 2 – Launch an EC2 Instance and Add User Data Script

Step 1: Launch an EC2 Instance and Attach the Security Group

1. Go to EC2 > Instances > Launch instances.



2. Choose an AMI: **Canonical, Ubuntu 24.04**.

3. Choose Instance Type: **t3.micro** (free-tier account) or another small type.

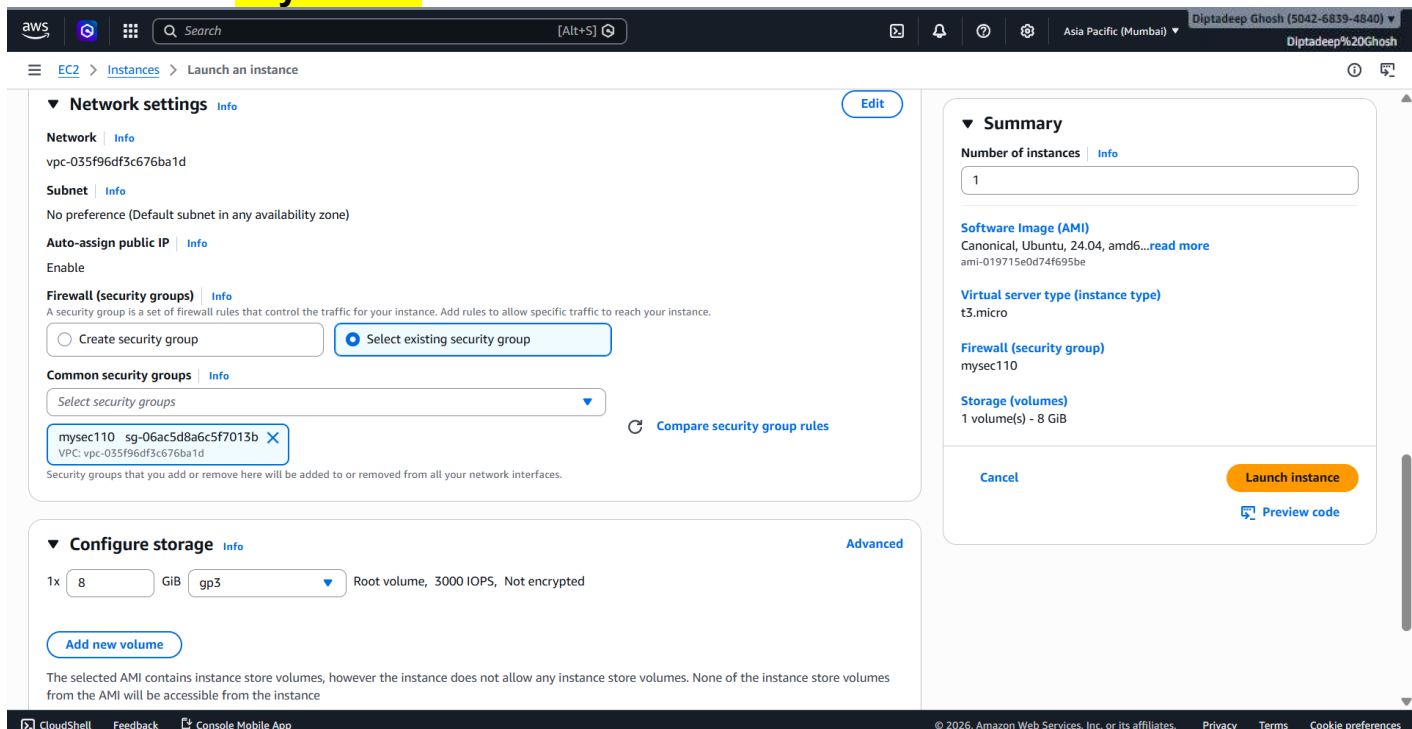
4. Create a key Pair in my case : **mysec110**.

5. Configure instance details:

- **Network (VPC):** choose the **same VPC** used for the security group.
- **Subnet:** choice **default**.
- **Auto-assign public IP:** **Enable**

6. Firewall (security groups): choose **Select existing security group**

7. **Common Security Group** : Select the newly created Security Group. In my case its - **mysec110**.



8. Scroll to 'Advanced Details' section.

Step 2: Add User Data Script

1. Click on **Advance Details** .
2. **User data:** In that, paste the following script: (below). This runs on first boot and prepares the instance.

paste in the *User data* box:

```
#!/bin/bash
apt-get update
apt-get upgrade -y
apt-get install -y nginx
systemctl start nginx
systemctl enable nginx
apt-get install -y git
curl -sL https://deb.nodesource.com/setup_18.x | sudo -E bash -
apt-get install -y nodejs
```

For V2 requests, you must include a session token in all instance metadata requests. Applications or agents that use V1 for instance metadata access will break.

Metadata response hop limit [Info](#)

2

Allow tags in metadata [Info](#)

Select

User data - optional [Info](#)

Upload a file with your user data or enter it in the field.

[Choose file](#)

```
#!/bin/bash
apt-get update
apt-get upgrade -y
apt-get install -y nginx
systemctl start nginx
systemctl enable nginx
apt-get install -y git
curl -sL https://deb.nodesource.com/setup_18.x | sudo -E bash -
apt-get install -y nodejs
```

Summary

Number of instances [Info](#)

1

Software Image (AMI)

Canonical, Ubuntu, 24.04, amd64...[read more](#)

ami-019715e0d74f695be

Virtual server type (instance type)

t3.micro

Firewall (security group)

mysec110

Storage (volumes)

1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

3. Click 'Launch instance'.

Success

Successfully initiated launch of instance (i-058ca112d31e231ec)

Launch log

Next Steps

What would you like to do next with this instance, for example "create alarm" or "create backup"

Create billing usage alerts

To manage costs and avoid surprise bills, set up email notifications for billing usage thresholds.

[Create billing alerts](#)

Connect to your instance

Once your instance is running, log into it from your local computer.

[Connect to instance](#) [Learn more](#)

Connect an RDS database

Configure the connection between an EC2 instance and a database to allow traffic flow between them.

[Connect an RDS database](#) [Create a new RDS database](#) [Learn more](#)

Create EBS snapshot policy

Create a policy that automates the creation, retention, and deletion of EBS snapshots

[Create EBS snapshot policy](#)

Manage detailed monitoring

Enable or disable detailed monitoring for the instance. If you enable detailed monitoring, the Amazon EC2 console displays monitoring graphs with a 1-minute period.

Create Load Balancer

Create an application, network gateway or classic Elastic Load Balancer

[Create Load Balancer](#)

Create AWS budget

AWS Budgets allows you to create budgets, forecast spend, and take action on your costs and usage from a single location.

[Create AWS budget](#)

Manage CloudWatch alarms

Create or update Amazon CloudWatch alarms for the instance.

[Manage CloudWatch alarms](#)

PHASE 3 – Connect to the Instance (EC2 Instance Connect)

Step 1: Connect to EC2 Instance

1. In the **Instances** list, select the newly launched instance.

The screenshot shows the AWS Management Console interface. On the left is a navigation menu with categories like EC2, Images, and Elastic Block Store. The main area displays the 'Instance summary for i-058ca112d31e231ec (mysec110(instance))'. The instance is in a 'Running' state. Key details include: Public IPv4 address 15.206.70.197, Private IPv4 address 172.31.37.239, and Public DNS ec2-15-206-70-197.ap-south-1.compute.amazonaws.com. There are buttons for 'Connect', 'Instance state', and 'Actions'.

2. Click **Connect > EC2 Instance Connect**.

3. Choose **Connect using a Public IP** (default). The **Username** for the Ubuntu AMI is **ubuntu**.

4. Click **Connect** — It will open a browser-based terminal session.

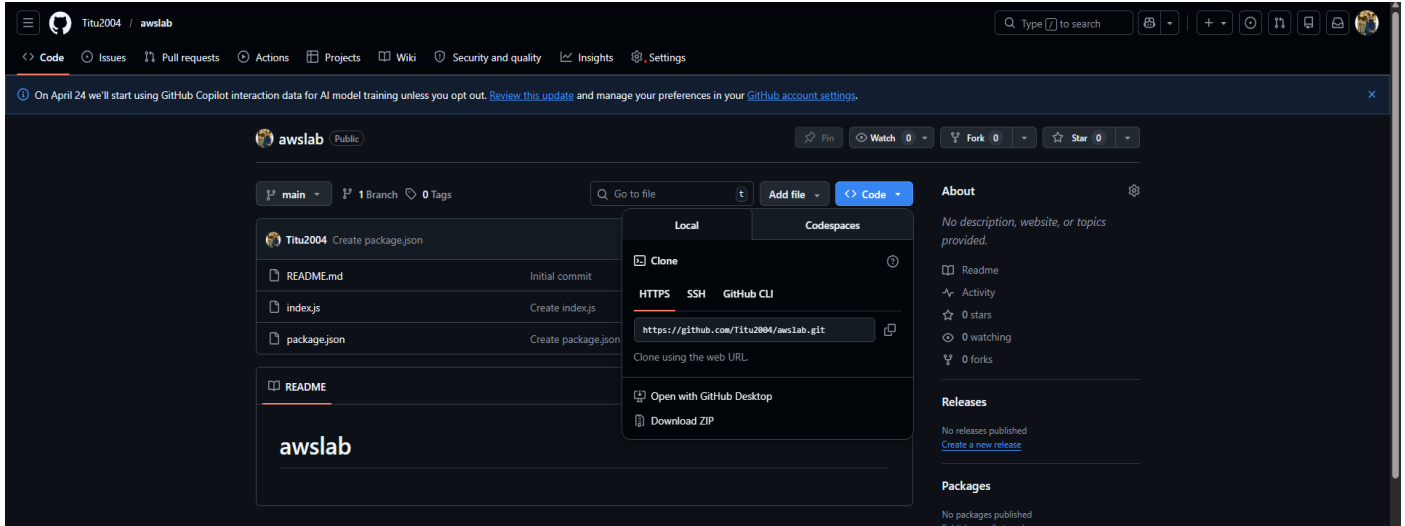
The screenshot shows the 'Connect to instance' page in the AWS Management Console. It provides details for instance i-058ca112d31e231ec, including VPC ID, Security groups, and IAM role. Below this, there are tabs for 'EC2 Instance Connect', 'SSM Session Manager', 'SSH client', and 'EC2 serial console'. The 'EC2 Instance Connect' tab is active, showing options to 'Connect using a Public IP' (selected) or 'Connect using a Private IP'. Under the 'Public IP' section, the 'Public IPv4 address' 15.206.70.197 is listed. The 'Username' field is set to 'ubuntu'. A note at the bottom states: 'Note: In most cases, the default username, ubuntu, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.' At the bottom right are 'Cancel' and 'Connect' buttons.

(Then Its show successful connection and the **public IP: 15.206.70.197** and username **ubuntu**.)

PHASE 4 – Pull the Project from GitHub and Start the Website

Step 1: Clone the REPO From GITHUB

1. Go to **GITHUB Account** .
2. Open the which have node files , in my case '**awslab**' , and **copy** the REPO URL Link.



3. Then, Once connected via Instance Connect, **perform these commands** in the **terminal**:

For clone the GitHub repo used

git clone <https://github.com/Titu2004/awslab.git>

To change the directory & for checking the files that clone actually done

cd aws
ls

It will show

index.js package.json

To Install Node dependencies

npm install

To Start the Node app

node index.js

It will show Result as: **"Started server"**


```

remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (14/14), done.
remote: Total 15 (delta 5), reused 4 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (15/15), 4.15 KiB | 1.38 MiB/s, done.
Resolving deltas: 100% (5/5), done.
ubuntu@ip-172-31-37-239:~$ cd awslab
ubuntu@ip-172-31-37-239:~/awslab$ ls
index.js  package.json
ubuntu@ip-172-31-37-239:~/awslab$ npm install

added 224 packages, and audited 225 packages in 23s

26 packages are looking for funding
  run `npm fund` for details

1 low severity vulnerability

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
npm notice
npm notice New major version of npm available! 10.8.2 -> 11.10.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.10.1
npm notice To update run: npm install -g npm@11.10.1
npm notice
ubuntu@ip-172-31-37-239:~/awslab$ node index.js
Started server

```

i-058ca112d31e231ec (mysec110(instance))

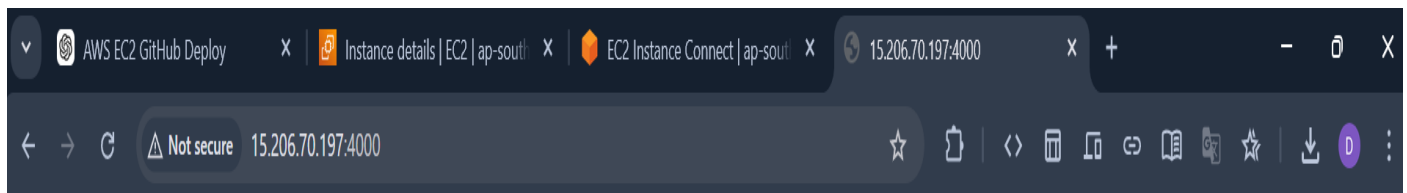
PublicIPs: 15.206.70.197 PrivateIPs: 172.31.37.239

Step 2: Verify This From MY Browser

1. Copy the **Public IPv4 address** from **EC2 Instance dashboard**.
2. Check the default nginx page (HTTP port 80): open **http://<PUBLIC_IP>** (e.g., **http://15.206.70.197**) — we can see **Welcome to nginx!**



3. Check the Node.js app on port 4000: open **http://<PUBLIC_IP>:4000** (e.g., **http://15.206.70.197:4000**) — we can see the web output -> **Hello MCKV**



Hello MCKV

Output :

The project hosted on GitHub is successfully deployed on AWS

EC2 instance and accessible via public IP address.

Conclusion :

This lab demonstrates how to deploy a GitHub project on AWS EC2 by configuring a Security Group and using User Data automation script.